

Production-Level Backend System – Full Code Examples

This document provides **real-world production-ready code** examples addressing **common backend issues** such as sharding, caching, Kafka integration, and distributed consistency. All code snippets are combined in one place for reference.

1. Express.js Server with Sharded MongoDB Connection

```
// db.js - MongoDB Sharding Connection Manager
const { MongoClient } = require('mongodb');

const shardRegistry = {
  0: 'mongodb://shard0.example.com:27017',
  1: 'mongodb://shard1.example.com:27017',
  2: 'mongodb://shard2.example.com:27017',
};

const connections = {};

async function getShardConnection(userId) {
  const shardId = userId % Object.keys(shardRegistry).length;
  if (!connections[shardId]) {
    connections[shardId] = await MongoClient.connect(shardRegistry[shardId], {
      useUnifiedTopology: true });
  }
  return connections[shardId].db('app');
}

module.exports = { getShardConnection };
```

```
// server.js - Express API with Sharded DB and Redis Cache
const express = require('express');
const Redis = require('ioredis');
const { getShardConnection } = require('./db');

const app = express();
const redis = new Redis();
app.use(express.json());

// Get user info with cache and sharded DB
```

```

app.get('/user/:id', async (req, res) => {
  const userId = parseInt(req.params.id);
  const cacheKey = `user:${userId}`;

  // Try Redis cache first
  let user = await redis.get(cacheKey);
  if (user) return res.json(JSON.parse(user));

  // Fetch from correct shard
  const db = await getShardConnection(userId);
  user = await db.collection('users').findOne({ userId });

  if (user) {
    await redis.set(cacheKey, JSON.stringify(user), 'EX', 300); // 5 min TTL
  }
  res.json(user || {});
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

Production Considerations: - Redis caching avoids frequent DB calls. - Sharding ensures scalability for billions of users. - TTL and granular keys prevent memory bloat.

2. Kafka Producer & Consumer for Event-Driven Architecture

```

// kafka.js
const { Kafka } = require('kafkajs');
const kafka = new Kafka({ clientId: 'task-service', brokers: ['broker1:9092', 'broker2:9092'] });
const producer = kafka.producer();
const consumer = kafka.consumer({ groupId: 'notification-group' });

module.exports = { kafka, producer, consumer };

```

```

// producer.js - Emit task events
const { producer } = require('./kafka');

async function emitTaskCreated(task) {
  await producer.connect();
  await producer.send({
    topic: 'task_created',
    messages: [{ key: `user_${task.userId}`, value: JSON.stringify(task) }],
  });
}

```

```
module.exports = { emitTaskCreated };
```

```
// consumer.js - Consume events
const { consumer } = require('./kafka');
const { sendEmailNotification } = require('./notification');

async function startConsumer() {
  await consumer.connect();
  await consumer.subscribe({ topic: 'task_created', fromBeginning: false });

  await consumer.run({
    eachMessage: async ({ message }) => {
      const task = JSON.parse(message.value.toString());
      await sendEmailNotification(task.userId, task.taskId);
    },
  });
}

startConsumer();
```

Production Considerations: - Kafka decouples services for async processing. - Consumer ensures idempotency by using `taskId` as key. - Replayable events handle downtime gracefully.

3. Redis Cache Invalidation via Kafka Event

```
// cacheInvalidation.js
const redis = new (require('ioredis'))();

async function invalidateUserCache(userId) {
  const cacheKey = `user:${userId}`;
  await redis.del(cacheKey);
}

module.exports = { invalidateUserCache };
```

```
// consumer.js snippet
const { invalidateUserCache } = require('./cacheInvalidation');

await consumer.run({
  eachMessage: async ({ message }) => {
    const event = JSON.parse(message.value.toString());
    await invalidateUserCache(event.userId); // Clear outdated cache
  }
});
```

```
    },  
  });  
};
```

Issue Solved: Cache consistency for user-specific updates without blocking DB writes.

4. Handling Distributed Transactions with Saga Pattern

```
// saga.js - Example of task creation with compensating actions  
async function createTaskSaga(task) {  
  try {  
    // Step 1: Create task in DB  
    const db = await getShardConnection(task.userId);  
    await db.collection('tasks').insertOne(task);  
  
    // Step 2: Emit Kafka event  
    await emitTaskCreated(task);  
  
    // Step 3: Update analytics asynchronously  
    // If analytics fails, it won't block the user (compensating actions handled  
    // separately)  
  } catch (err) {  
    // Compensating action: Rollback task creation if needed  
    console.error('Saga failed:', err);  
  }  
}
```

Production Considerations: - Distributed operations are decoupled for availability. - Compensating actions ensure eventual consistency. - Idempotency prevents duplicate side-effects.

5. Aggregation Pipeline Example for Analytics

```
// analytics.js  
async function getUserTaskStats(userId) {  
  const db = await getShardConnection(userId);  
  return db.collection('tasks').aggregate([  
    { $match: { userId } },  
    { $group: { _id: "$status", total: { $sum: 1 } } }  
  ]).toArray();  
}
```

Issue Solved: Efficient reporting on large datasets without cross-shard joins.

6. Production Best Practices Included in Code

- **Sharded DB** for scaling user data.
- **Redis caching** for frequently accessed data.
- **Kafka event-driven architecture** for decoupled services.
- **Saga pattern** for distributed transaction management.
- **Cache invalidation** to maintain consistency.
- **Aggregation pipelines** for analytics without overloading DB.
- **Idempotency** in consumers and cache writes.
- **Error handling & compensating actions** for production reliability.

This document contains **all production-ready backend code and patterns** in a single place for reference, ready for implementation in real-world distributed systems.